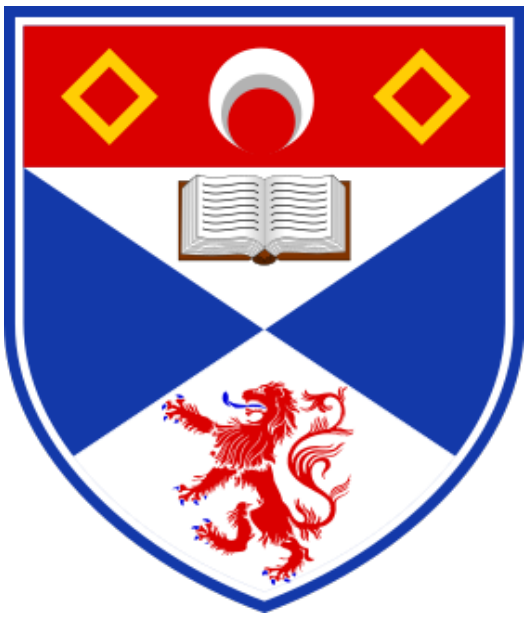
CS1006 - P3

Nonograms



Group 05: 240012803, 240016926, 24002265, 240011673

University of St Andrews
Friday 25th April, 2025

Contents

1 Overview **1**

2 Team Working **1**

3 Coding Design **1**

4 Testing **6**
4.1 Model Testing 12
4.2 Solver Testing 12

5 Evaluation and Conclusions **13**

6 References **13**

7 Appendices **13**

1 Overview

The aim of this practical was to create a Nonogram puzzle game and solver. We were able to fully accomplish primary and secondary requirements and create a partially successful deductive solver.

We designed a series of classes that represented elements of the game such as the grid board, a clue and a move to create a model that we could build on. On top of this model, we created a JSON parser to load the puzzle files, a UI to allow the user to interact with the puzzles, and a checker that flags incorrectly filled rows and columns. These elements satisfied the primary requirements. To satisfy the secondary requirements, we modified the model to allow the user to undo, save and load moves as well as play with colours.

We worked effectively as a group and extensively tested all methods to ensure user friendly error handling.

2 Team Working

We worked well together by meeting regularly, updating each other with our progress on a shared groupchat, and distributing the workload fairly. For the primary and secondary requirements, 24002265 designed the parser with integrated error handling, 240016926 designed a checker for the puzzles, 240012803 created the original model classes and wrote back-end methods to manipulate the model (e.g. record a move, save a grid etc.), while 240011673 implemented the UI in JavaSwing.

We set and met internal deadlines and coded our work allocation well. We used version control and initially created our own branches for each element we took a lead on to keep a neat workflow and minimize merge conflicts. 240012803 designed the original organization of the solver while the whole group worked collaboratively on the methods implementing it such as the possible row generation and searchspace weeding.

3 Coding Design

Model

The core component of the nonogram's model is the `Grid` object. It contains information such as the dimensions of the puzzle, the `Clues` for each row and column, a 2-D array of integers representing the

current puzzlegrid state and a record of all `Moves` the user has made.

Each `Clue` object contains two parallel ArrayLists, one to hold the size of each block of cells and the second to hold the integer representation of the colour corresponding to each block. Each `Move` object stores the location of the cell in the grid, the old and the new colour.

This object has two constructors; one for representing the information gathered from a puzzle with clues, and one for representing a user grid. The first constructor is used by the parser when creating a grid already populated by clues. In the user grid, as the user plays, each move will add a `Move` object to the `moves` ArrayList and the 2-D array is also updated in the backend. When the `loadGrid()` is called, the `moves` ArrayList is read from a Json file which, if read sequentially, will load the game to the current state in the order of it's play. To undo a move, `undoGrid()` uses the last move object to change the 2-D array, then removes the move from the ArrayList and finally updates the Json file of moves by writing the moves to the file using `saveGrid()`.

To check the two puzzles against each other, one could simply convert the user's in-progress grid into a series of clues and then check if they are the same as the player grid's clues.

Parser

The Parser takes the user-specified Json and creates array-lists corresponding to the rows, columns,, andcolors. A global hashmap stores numeric representations of the colours for a specified nonogram. `getArrayLists()` uses `clearLists()` to ensure empty Lists. Then the Json structure is created from the selected Json. This structure is then passed to `getArrayListsFromTree()`. Then if the colours hash-map is empty the unknown, empty and black colours are added respectively. `getArrayListsFromTree()` uses the Json structure created in `getArrayLists()` and navigates through it, taking the information we need from the Json. This was adapted from 19.3.3 [1]. The parser then generates an arraylist of clues for every row and column. The `checkNumColour()` method ensures that the Json has the correct amount of colours by creating a set of all the colours in the nonogram and comparing it to the length of the colours hashmap minus two for the unknown and empty colours.

Checker

This class was split into a number of different methods, to help neaten the code and keep it modular.

`checkNonogram()`
This method loops through the number of rows and columns and checks each row and column individually via the checkLine() method. This design was chosen as it keeps the code neater and more modular. The incorrect rows/columns were referred to be their index and added to a list of incorrect rows and incorrect columns, which were added to a separate list. This design was chosen as it allows the checkNonogram() method to return it, and it is easy to access the incorrect rows and columns this way.

`checkLine()`
This method compares a given line (either a row or a column) to a given Clue, and ensures the line meets the Clue's constraints. This approach allows us to call it within a loop of every row and column, keeping it neat and readable. If there is no given clue count, it is taken as '0' and therefore the line must not have any coloured squares, and this is checked at the beginning of this method. I decided to split the clue into its counts and corresponding colours: so a clue of 5, 3, 2 and colours 2, 3, 2 will be treated as (5, 2), (3, 3), (2, 2), with a variable to keep track of the position in the line, which was a neat and simple way to handle this. I also decided to allow for the user to not have to fill in the square as 'empty' and so 'unknown' will still be fine and the checker will work as expected. This design was chosen as it is a nice quality of life change.

`getColumn()`
This method was added as it allows for just one checkLine() method (opposed to checkRow() and checkColumn()). This helped keep the code more readable.

`getMessage()`
This method was added to neatly construct the checker message which is output to the user once their grid is checked, which keeps the code neater and modular. This method calls the getProgress() method which was added as a small bonus feature which helps give the user track how well they are doing. The calculated value was rounded to one decimal place [2].

UI

The UI is defined entirely in the class 'App.java', where components are created, and methods are called.

Main method
The main method, when run, creates a new object of the App class and creates a new JFrame within which the program will run. The size and other parameters for the window are then set using Java Swing methods.

It also handles when the user closes out of the game, it will prompt a save if not currently saved.

Constructor
Upon creating the new object, it does the following:

- Creates all UI components, including hotkeys and listeners to handle button clicks, through the setupUI() method.
- Open a screen asking the user if they want to play, or if they want to read the instructions, through the openGameLauncher() method.

`setupUI()`
This method makes use of Java Swing methods to create objects of different Swing components, such as JPanels, JButtons, and JLabels. It is clearly presented with code separated by which section of the screen they refer to. This code also contains action listeners, code that is run when a button is clicked. We spent time refactoring this code multiple times to change how components were presented on the screen through different layout modes. At first most of my components used GridLayout but this proved difficult for components that needed to be resized, or specifically only parts of them resized. The UI uses a variety of layout options such as GridLayout, FlowLayout, BoxLayout, and GridBagLayout. There is a helper method createStyledButton() which styles each of the buttons in the control panel at the bottom of the screen.

`openGameLauncher()`
This method contains Java Swing code to display a small welcome message on the screen, prompting the user to either start playing or reading the instructions.

If the user opts to start playing instantly, the loadGamePuzzle() method is called. (see below).

If the user opts to read the instructions, a small dialogue box appears with instructions on how to play the game as well as useful information about hotkeys.

If the user clicks the exit icon on the window, the program stops running.

`loadPuzzleGame()`
This method gets the puzzle name and grid name from the user to load. It gets these from two dropdown menus, and an additional input dialog for the grid name if the user is creating a new grid.

It then creates the colours and builds the grid (see below)

Cases where the user exits out the menu are handled, as well as any exceptions while parsing JSON are thrown to this method and are displayed on screen, for the user to then select a new puzzle.

`displayColors()`
This method creates a new colour button for each colour in the hashmap.

The action listener will highlight the currently selected button and change the currentlySelectedColour variable to match the key of the hashmap. This means that whenever a user clicks on a cell to paint it, it will apply the currently selected colour. We initially had it so clicking a cell would iterate through colours but this is inefficient and tedious for the user.

Each colour also has a related key shortcut on the keyboard that is one more than its hashmap key. This is so the user doesn't click 0 for unknown as it is far away from 1,2,3 etc on the keyboard.

Each cell is given unknown as it's starting value, which is stored as a property as part of the cell using the hashmap key.

`buildGrid()`
This is where the majority of the code for the UI is located. This method builds the entire grid as well as clues. We spent a considerable amount of time changing our approach to how clues were displayed. Originally it was two panels, one above and one to the left of the grid, that contained the clues. However, this presented many alignment issues.

Our final solution was to implement all of the clues into the grid as an additional row and column which did take substantial refactoring of other methods to ensure they did not account for the first row and column.

The source variable ensures that the handling of any 'Cancel' buttons are done correctly. If the source is fromStart, clicking cancel on any dropdown menu will close the entire program, however if it is fromButton, it just closes the dropdown window. This is because if the user goes to load a file from the button while in game and then decides to cancel, we do not want the game completely closing.

The method starts by initialising an empty cell in the top left of the grid, adds the clues, and then the rest of the grid.

The rest of the grid is then built on screen with the correct colour. Every 5 rows and columns, the border is made thicker to improve readability.

The final piece of the puzzle is to add the listener, which is triggered when a cell is clicked. Additionally, we have implemented functionality for dragging so users can fill in larger parts of the puzzle easier.

`cellClicked()`
This method applies the currently selected colour to the cell, by changing the background colour of the cell as well as the cells state property, which holds the hashmap key of the colour that is currently applied.

`restartApp()`
When a user chooses a file that throws an error, the error is displayed, and then this function is called to allow the user to go back and select another puzzle.

`allowUndo(), disableUndo()`
These methods toggle if the user is able to undo or not. If the user is refrained from undo-ing, the button is grey-ed out and disabled. This change is then reverted if they are allowed to undo. The scenario in which this would occur is if the user has clicked the undo button enough times to take them back to a blank grid, in which there are no moves for them to undo.

A user being allowed to undo means that there are more than 0 objects within the grid's json file containing moves.

`allowSave(), disableSave()`
These methods toggle if the user is allowed to save or not. If the user is refrained from saving, the button is grey-ed out and disabled. This change is then reverted if they are allowed to save.

As soon as the user saves, saving becomes disabled until they make another move. This is handled in each cell's action listener.

`askSave()`
A simple method that presents the user with a message asking if they want to save their progress. This is called when the user goes to load a new puzzle through the button or when they try to close the game, and their current puzzle is not saved.

`saveGrid()`
This method is called by clicking the 'Save' button, or clicking 'Yes' to any dialog asking to save.

`solvePuzzle()`
This method is called by clicking the 'Solve' button, and asks the user for confirmation, then solves the puzzle.

`showTemporaryText()`
This method changes the text label in the control panel to show a message for 3 seconds. This message is used for when the user Saves, Clears Grid, or completes the puzzle.

Solver

The solver creates a search space using the classes Arrangements and Nodes. Each node contains three parallel ArrayLists, one representing the first index of each clues placement, one representing the length of each clue and a third representing the colors of each clue. This being done, a unique row/column can be generated from each node. An array of every possible state for a row or column is stored in an arrangement object.

To generate all the arrangements we used the getPlaces() method. This loops through each possible placement for a singular clue, then expands on it by creating a new array of arrays of clue positions based on the previous clues. This required creating temporary arrays to prevent exceptions being thrown due to concurrent array modification. This method returns an arraylist of arraylist of integers that can be looped through to generate all potential nodes for a line. A snippet of the code can be seen below.

To optimize the solver we started by checking if any rows only had one possibility and if they did that would get filled in, this is done in `solver()`. Then we would check the rows and columns with single clues for any overlap and then fill the overlap in, this was split into 2 methods `optimizeRow()` and `optimizeCol`, these were then called in `optimize()`. We were going to add this overlap for every type of clue however, we ran out of time.

The deductive solver loops through and deletes invalid arrangements until only one left then adds this arrangement to the grid filling in the associated filled and blank cells.

Further Features

In addition to all the classes and methods above, we have a `README` file that contains instructions on how to run our game, and the testing methods too.

```
1     public ArrayList<ArrayList<Integer>> getplaces(ArrayList<Integer> blocks, int index,
          int length, ArrayList<Integer> originalPlace, int place){
2         ArrayList<ArrayList<Integer>> total = new ArrayList<>();
3         while (index >= 0) {
4             // gets the first possible place of the current block
5             place = originalPlace.get(index);
6             ArrayList<Integer> places = new ArrayList<>();
7             // will loop through the potential placement for the blocks
8             while (place + blocks.get(index) <= length ) {
9                 places.add(place);
10                place++;
11            }
12            if (total.size() == 0) {
13                for (int i : places) {
14                    ArrayList<Integer> temp = new ArrayList<>();
15                    temp.add(i);
16                    total.add(temp);
17                }
18            }
19            else {
20                ArrayList<ArrayList<Integer>> tempTotal = new ArrayList<>();
21                for (ArrayList<Integer> combo : total) {
22                    for (Integer i : places) {
23                        if (i + blocks.get(index) <= combo.get(0)) {
24                            ArrayList<Integer> temp = new ArrayList<>(combo);
25                            temp.addFirst(i);
26                            tempTotal.add(temp);
27                        }
28                    }
29                }
30                total = new ArrayList<>(tempTotal);
31            }
32            length = places.getLast();
33            index --;
34        }
35        return total;
36    }
```

Figure 1: Node permutation generation

4 Testing

Checker Testing

The checker was tested by creating a new class, TestChecker.java, which tested the different methods in the Checker.java class. The preconditions for each test (the 2D integer array grids and the ArrayList of clues) were created by calling methods in the TestChecker class to keep the code neater. The results of these tests were validated by the checkTestStatus() method and the results were output to the terminal. All of the 24 tests passed and the results were as expected.

Table 1: checkNonogram() and getMessage() [Tests 1 to 11]. Both of these methods are tested per test case for readability and to reduce the class length.

Test case and explanation	Pre-conditions	Expected outcome	Actual outcome

If the checker identifies a correct blanks_smiler grid (and the other cells are 'unknown'), by checking if the ArrayLists of Integers returned are empty (so there are no incorrect lines) and the message is correct.	The solved grid, row clues and column clues for blanks_smiler were created.	Two empty lists and the message to say the grid is fully completed.	`[][]`
			`All rows are correct.`
			`All columns are correct.`
			`Completion: 100.0%`

If the checker identifies a correct blanks_smiler grid (and the other cells are 'empty'), by checking if the ArrayLists of Integers returned are empty, and the message is correct.	The solved grid, row clues and column clues for blanks_smiler were created.	Two empty lists and the message to say the grid is fully completed.	`[][]`
			`All rows are correct.`
			`All columns are correct.`
			`Completion: 100.0%`

If the checker correctly handles the unsolvable_smiler by returning the error in column 10.	The attempted solved grid, row clues and column clues for unsolvable_smiler.	An empty row list but the column list should have the incorrect column (which is column index 9), and the correct corresponding message.	`[][9]`
			`All rows are correct.`
			`Incorrect column 10`
			`Completion: 95.0%`

If the checker correctly handles the unsolvable_smiler where the user has filled in the final column but in return has violated the final row's clue.	The attempted solved grid, row clues and column clues for unsolvable_smiler.	An empty column list but the row list should have the incorrect row (index 9) and the correct corresponding message.	`[9][]`
			`All columns are correct.`
			`Incorrect row 10`
			`Completion: 95.0%`

If the checker identifies a correct colour_wink grid.	The solved grid, row clues and column clues for colour_wink.	Two empty lists and the message to say the grid is fully completed.	`[][]`
			`All rows are correct.`
			`All columns are correct.`
			`Completion: 100.0%`

Table 2: checkLine() [Tests 12 to 21]

If the checker identifies a correct multi_checks grid. There are two possible solutions to this grid, and so this tests if the checker can handle both (the other solution is handled in the test case below this one).	The solved grid, row clues and column clues for multi_checks.	Two empty lists and the message to say the grid is fully completed.	[][] All rows are correct. All columns are correct. Completion: 100.0%

If the checker identifies a different colour multi_checks grid.	The solved grid (a different solved grid to the one above), row clues and column clues for multi_checks.	Two empty lists and the message to say the grid is fully completed.	[][] All rows are correct. All columns are correct. Completion: 100.0%
If the checker identifies a correct colour_cat grid. This grid has multiple different colours.	The solved grid, row clues and column clues for colour_cat.	Two empty lists and the message to say the grid is fully completed.	[][] All rows are correct. All columns are correct. Completion: 100.0%
If the checker correctly identifies an error for a particular grid, correctly identifying the row/s and column/s it appeared in and have this be included in the message.	An incorrect grid which has one error, row clues and column clues for cat.	The row list will have one element (1, which is the index of row 2) and the column list will have one element (6, which is the index of row 7), and the correct error message.	[1][6] Incorrect row 2 Incorrect column 7 Completion: 90.0%
If the checker correctly identifies multiple errors for a particular grid, correctly identifying the rows and columns the errors appeared in as well as showing this in the message.	An incorrect grid which has multiple errors, row clues and column clues for cat.	The row list and column list will have 4 elements each inside of them, which are the indices of where the errors occurred.	[0, 3, 7, 9][2, 4, 5, 8] Incorrect row 1 Incorrect row 4 Incorrect row 8 Incorrect row 10 Incorrect column 3 Incorrect column 5 Incorrect column 6 Incorrect column 9 Completion: 60.0%

If the checker correctly identifies an empty grid as completely incorrect, correctly showing this in the returned lists and message.	A completely blank grid (i.e. filled with unknowns and empties, no colours), row clues and column clues for cat.	The row list and column list will have 10 elements each, of every row and column (indices 0-9), and the correct corresponding message.	`[0, 1, 2, 3, 4, 5, 6, 7, 8, 9][0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`
			`Incorrect row 1`
			`Incorrect row 2`
			`Incorrect row 3`
			`Incorrect row 4`
			`Incorrect row 5`
			`Incorrect row 6`
			`Incorrect row 7`
			`Incorrect row 8`
			`Incorrect row 9`
			`Incorrect row 10`
			`Incorrect column 1`
			`Incorrect column 2`
			`Incorrect column 3`
			`Incorrect column 4`
			`Incorrect column 5`
			`Incorrect column 6`
			`Incorrect column 7`
			`Incorrect column 8`
			`Incorrect column 9`
			`Incorrect column 10`
			`Completion: 0.0%`

Test case and explanation	**Pre-conditions**	**Expected outcome**	**Actual outcome**
If the checkLine() method correctly identifies a wrong line (i.e. line does not meet the clue constraint) where the line has the incorrect number of counts/cells (6 instead of 5).	A line (an integer array) which has 6 squares, and a Clue object which has count 5. Colour 2 has been used in this case.	False should be returned since the line is incorrect.	`false`
If the checkLine() method correctly identifies a wrong line where the line has the incorrect count but the same number of cells (2, 3 instead of 5).	A line (an integer array) which has 2 squares, a space (unknown), and 3 squares, and a Clue object which has count 5. Colour 2 has been used in this case.	False should be returned since the line is incorrect.	`false`
If the checkLine() method correctly identifies a wrong line where the line has the correct count/number of cells but the incorrect colour (colour 3 instead of 2).	A line (an integer array) which has 5 squares in colour 3, and a Clue object which has count 5 in colour 2.	False should be returned since the line is incorrect.	`false`
If the checkLine() method correctly identifies a wrong line where the line has both the incorrect count and colour (2, 3 in colour 3 instead of colour 2).	A line (an integer array) which has 2 squares in colour 3, a space (unknown), and 3 squares in colour 3, and a Clue object which has count 5 in colour 2.	False should be returned since the line is incorrect.	`false`
If the checkLine() method correctly identifies a wrong line where the line has the correct number of cells and colour but the wrong count	A line (an integer array) which has 6 squares in colour 2, and a Clue object which has count 4, 2 in colour 2	False should be returned since the line is incorrect.	`false`

If the checkLine() method identifies a correct line which meets the constraint (3 cells in colour 2 and 2 cells in colour 3).	A line (an integer array) which has 3 squares in colour 2 and 2 squares in colour 3, and a Clue object which has count 3, 2 in colours 2, 3.	True should be returned since the line is correct.	`true`
If the checkLine() method identifies a correct line which meets the constraint (3 cells in colour 2 and 2 cells in colour 3). This time the line has an (optional) space between the counts.	A line (an integer array) which has 3 squares in colour 2, a space (an empty cell), and 2 squares in colour 3, and a Clue object which has count 3, 2 in colours 2, 3.	True should be returned since the line is correct.	`true`
If the checkLine() method identifies a correct line which meets the constraint (3 cells in colour 2 and 2 cells in colour 3). This time, the line has multiple (optional) spaces between the counts. This shows the checker can identify multiple variations of a correct line.	A line (an integer array) which has 3 squares in colour 2, 3 spaces (an unknown, an empty, an unknown), and 2 squares in colour 3, and a Clue object which has count 3, 2 in colours 2, 3.	True should be returned since the line is correct.	`true`
If the checkLine() method identifies a correct line which meets the constraint (2, 3 cells in colour 2). This shows that they must be separated by at least one space (unknown/empty) to be considered correct.	A line (an integer array) which has 2 squares in colour 2, a space (an unknown), and 3 squares in colour 3, and a Clue object which has count 2, 3 in colours 2, 2.	True should be returned since the line is correct.	`true`

Table 3: getColumn() [Test 22] and getProgress() [Test 23 and 24]

Test case and explanation	Method being tested	Pre-conditions	Expected outcome	Actual outcome

If the getColumn() method successfully returns the correct columns from the 2D array.	getColumn()	A 2D integer array was created.	The correct columns as an integer array should be returned.	`[0, 2, 4, 6, 8]`
				`[1, 3, 5, 7, 9]`
				`[5, 4, 3, 2, 1]`

If the getProgress() method successfully calculates how much the player has completed as a percentage.	getProgress()	Incorrect row and columns, and total rows and columns. All of these are integers.	The correct progress as a double should be returned.	`50.0`
If the getProgress() method successfully calculates how much the player has completed as a percentage, rounded correctly to one decimal place.	getProgress()	Incorrect row and columns, and total rows and columns. All of these are integers.	The correct progress as a double, rounded to one decimal place, should be returned.	`80.8`

Evidence of terminal output of the test cases are in the appendices section [Figures 1.1 - 1.4].

Parser Testing

The parser was tested using a new class, TestParser.java, which tested the different methods in the parser. There were 5 test cases and to check the methods the expected output was manually created and compared to the method output. All 5 tests passed with expected output.

Table 1: getClue() and getClues() [Tests 1 to 5]

Test case and explanation	Pre-conditions	Expected outcome	Actual outcome
Testing clue generation to ensure clues are generated as expected	2 arrayLists of Integers with the same amount of elements and a Clue created from these arrayLists	Clue created by getClue() is the same as the hardcoded and a message to terminall saying Test 1 Passed	Test 1 Passed
Testing that it fails when a Clue cant be generated from in-equal arraylist lengths	2 arrayLists of elements and a Clue created from these arrayLists	JsonFormatExceptio n trhown and Test 2 Passed sent to terminal	Test 2 Passed
Testing clue generation when arraylists are empty	2 empty araylists and a clue created from them	Clue is created and Test 3 Passed output to terminal	Test 3 Passed

Test case and explanation	Pre-conditions	Expected outcome	Actual outcome
Testing if getClue works with expected input	An arrayList of Clues generated from two ArrayList of ArrayList of integer, lines and colours.	Clues created and all match expected	Test 1 Passed
Testing it works if Lines and colours are empty	An arrayList of Clues generated from two empty ArrayList of ArrayList of integer, lines and colours.	Clues created and match expected	Test 2 Passed

The parser was also tested with a variety of corrupted and edge case Jsons to ensure only valid Jsons are accepted. There were 10 test cases and all passed with the expected Jsons parsing and corrupted ones failing. Some of these test cases are shown in the appendices section [Figures 2.1 - 2.4].

4.1 Model Testing

Tests for the back-end manipulations of the model can be found in `TestModel.java`. They print out the backend user grid as it is modified using the different backend methods.

4.2 Solver Testing

The testing for the solver is shown through videos in the /Videos folder. Specifically, videos 6,8, and 9. The testing clearly shows the solver completing the puzzles and displaying them on the screen. Unfortunately we could only get the deductive part of our solver to work, and only for specific puzzles. Those puzzles are listed as an array in `App.java` under `compatiblePuzzles`.

Front-end Testing

The frontend testing contains videos showcasing features of the UI. These videos can be found in /Videos, and there is a file in there containing information about each video: what is shows, and what it achieves/relates to expected output.

Additionally, there are some screenshots for further testing, which show the intended functionality, in the appendices section [Figures 3.1 - 3.8].

5 Evaluation and Conclusions

Overall, we successfully created a Nonogram puzzle with a GUI using the Java Swing libraries. We met the primary and secondary requirements to the extent outlined in the specification, as well as implementing some additional functionality for a better overall user experience. A partial solution was also created for the tertiary requirements. We effectively met the aims of the assignment by putting our CS1002 and CS1003 skills to the test as well as effectively working as a group. We also debugged, tested, and commented our code. The different features are neatly separated into different classes with their own methods and constructors. Given more time we would have liked to refine our Solver to make it work as intended, and also implementing the guessing function of the solver.

6 References

[1] https://docs.oracle.com/javaee/7/tutorial/jsonp003.htm (Navigating an object model)

[2] https://www.baeldung.com/java-round-decimal-number (”2. The Math.round() Method”)

7 Appendices

Checker tests evidence

Figure 1.1 Checker tests 1 - 6:

```
TEST ONE: Player has the correct blanks_smiler grid (All constraints met) and the rest of the cells are marked as 'unknown'.
TEST OUTPUT:
[][]
All rows are correct.
All columns are correct.
Completion: 100.0%
TEST STATUS: Pass

TEST TWO: Player has the correct blanks_smiler grid (All constraints met) and the rest of the cells are marked as 'empty'.
TEST OUTPUT:
[][]
All rows are correct.
All columns are correct.
Completion: 100.0%
TEST STATUS: Pass

TEST THREE: Player has attempted to solve the unsolvable_smiler.
TEST OUTPUT:
[][9]
All rows are correct.
Incorrect column 10
Completion: 95.0%
TEST STATUS: Pass

TEST FOUR: Player has attempted to solve the unsolvable_smiler and has (row 10, column 10) filled.
TEST OUTPUT:
[9][]
All columns are correct.
Incorrect row 10
Completion: 95.0%
TEST STATUS: Pass

TEST FIVE: Player has the correct colour_wink grid (All constraints met).
TEST OUTPUT:
[][]
All rows are correct.
All columns are correct.
Completion: 100.0%
TEST STATUS: Pass

TEST SIX: Player has the correct multi_checks grid (All constraints met).
TEST OUTPUT:
[][]
All rows are correct.
All columns are correct.
Completion: 100.0%
TEST STATUS: Pass
```

Figure 1.2 Checker tests 7 - 10:

```
TEST SEVEN: Player has the correct multi_checks grid (All constraints met).
TEST OUTPUT:
[][]
All rows are correct.
All columns are correct.
Completion: 100.0%
TEST STATUS: Pass

TEST EIGHT: Player has the correct colour_cat grid (All constraints met).
TEST OUTPUT:
[][]
All rows are correct.
All columns are correct.
Completion: 100.0%
TEST STATUS: Pass

TEST NINE: Player has an incorrect cat grid with one error in row 2, column 7.
TEST OUTPUT:
[1][6]
Incorrect row 2
Incorrect column 7
Completion: 90.0%
TEST STATUS: Pass

TEST TEN: Player has an incorrect cat grid with multiple errors in rows 1, 4, 8, 10 and columns 3, 5, 6, 9.
TEST OUTPUT:
[0, 3, 7, 9][2, 4, 5, 8]
Incorrect row 1
Incorrect row 4
Incorrect row 8
Incorrect row 10
Incorrect column 3
Incorrect column 5
Incorrect column 6
Incorrect column 9
Completion: 60.0%
TEST STATUS: Pass
```

Figure 1.3 Checker tests 11 - 18:

```
TEST ELEVEN: Player has an incorrect cat grid which is completely blank (filled with unknown and empty cells only). All constraints failed.
TEST OUTPUT:
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9][0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Incorrect row 1
Incorrect row 2
Incorrect row 3
Incorrect row 4
Incorrect row 5
Incorrect row 6
Incorrect row 7
Incorrect row 8
Incorrect row 9
Incorrect row 10
Incorrect column 1
Incorrect column 2
Incorrect column 3
Incorrect column 4
Incorrect column 5
Incorrect column 6
Incorrect column 7
Incorrect column 8
Incorrect column 9
Incorrect column 10
Completion: 0.0%
TEST STATUS: Pass

TEST TWELVE: The line has 6 squares instead of 5 like the clue states. False should be returned since the line is wrong.
TEST OUTPUT: false
TEST STATUS: Pass

TEST THIRTEEN: The line has 2 squares, a space (unknown), and 3 squares instead of 5 like the clue states.
TEST OUTPUT: false
TEST STATUS: Pass

TEST FOURTEEN: The line has 5 squares in colour 3 instead of in colour 2 like the clue states.
TEST OUTPUT: false
TEST STATUS: Pass

TEST FIFTEEN: The line has 2 squares in colour 3, a space (unknown), and 3 squares in colour 3 instead of 5 squares in colour 2 like the clue states.
TEST OUTPUT: false
TEST STATUS: Pass

TEST SIXTEEN: The line has 6 squares instead of 4 squares, a space, 2 squares like the clue states.
TEST OUTPUT: false
TEST STATUS: Pass

TEST SEVENTEEN: The line has 4 squares in colour 2, which meets the constraint.
TEST OUTPUT: true
TEST STATUS: Pass

TEST EIGHTEEN: The line has 3 squares in colour 2 and 2 squares in colour 3, which meets the constraint.
TEST OUTPUT: true
TEST STATUS: Pass
```

Figure 1.4 Checker tests 19 - 24:

```
TEST NINETEEN: The line has 3 squares in colour 2, a space (empty) and 2 squares in colour 3, which meets the constraint.
TEST OUTPUT: true
TEST STATUS: Pass

TEST TWENTY: The line has 3 squares in colour 2, 3 spaces (unknown and empty cells only) and 2 squares in colour 3, which meets the constraint.
TEST OUTPUT: true
TEST STATUS: Pass

TEST TWENTY ONE: The line has 2 squares in colour 2, 1 unknown, and 1 square in colour 2, which meets the constraint.
TEST OUTPUT: true
TEST STATUS: Pass

TEST TWENTY TWO: Tests the columns are successfully extracted from the 2D array grid.
TEST OUTPUT:
[0, 2, 4, 6, 8]
[1, 3, 5, 7, 9]
[5, 4, 3, 2, 1]
TEST STATUS: Pass

TEST TWENTY THREE: The progress for 16 total lines where 8 are incorrect (and so 8 are correct).
TEST OUTPUT: 50.0
TEST STATUS: Pass

TEST TWENTY FOUR: The progress for 26 total lines where 5 are incorrect (and so 21 are correct).
TEST OUTPUT: 80.8
TEST STATUS: Pass
```

Parser tests evidence

Figure 2.1: Testing 1x1 grid constraint

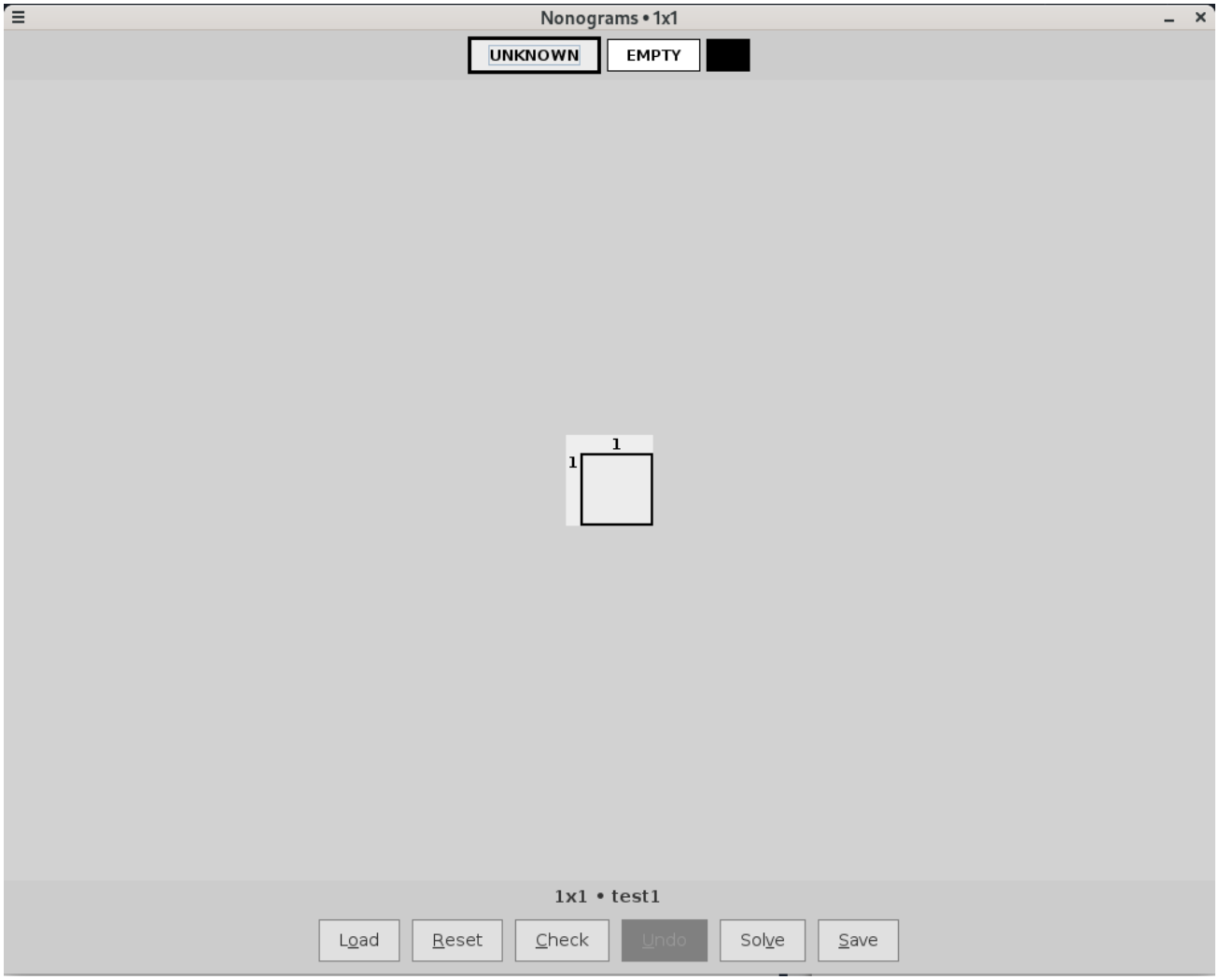


Figure 2.2: Testing 0x0 grid constraint

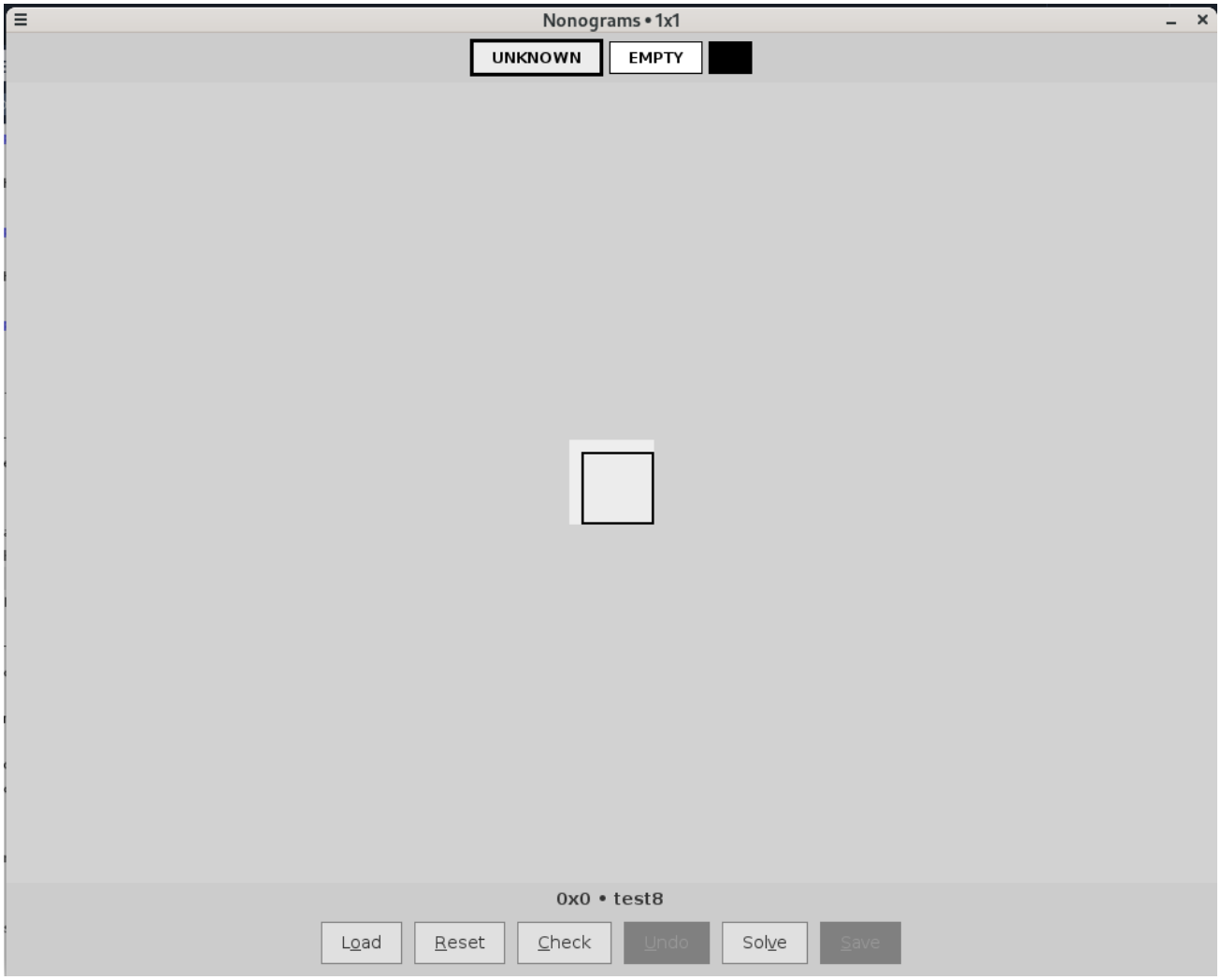


Figure 2.3: testing unsolvable grid constraint

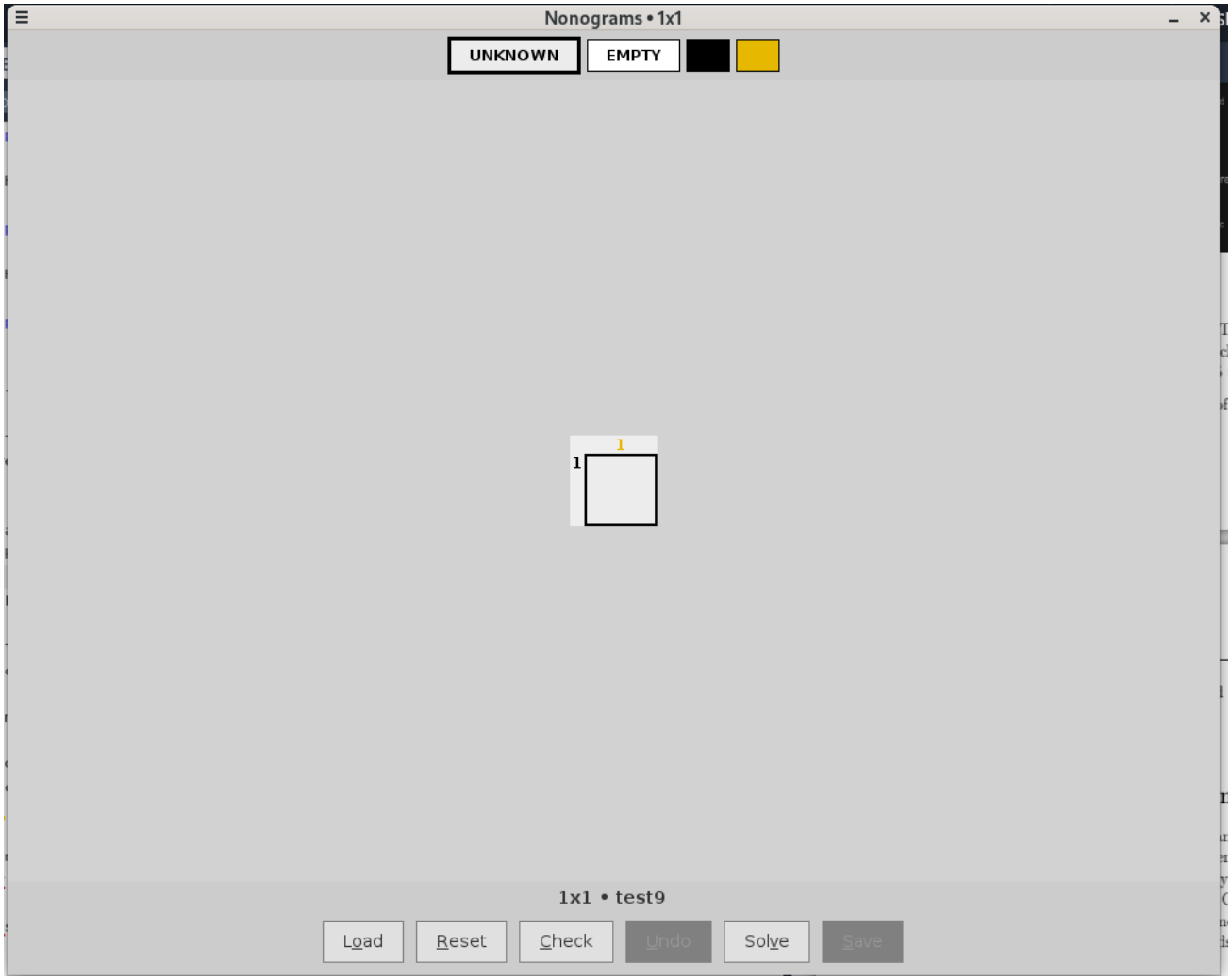
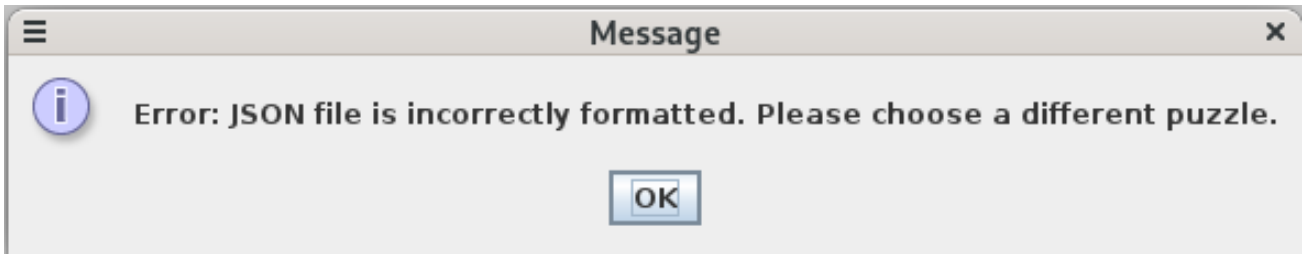


Figure 2.4: Error message for Invalid Jsons



GUI tests evidence

Figure 3.1: Start Screen

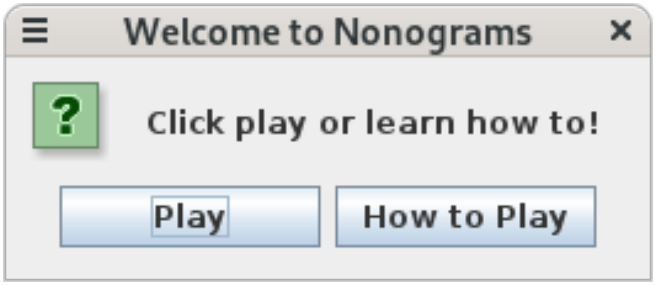


Figure 3.2: Instructions Screen

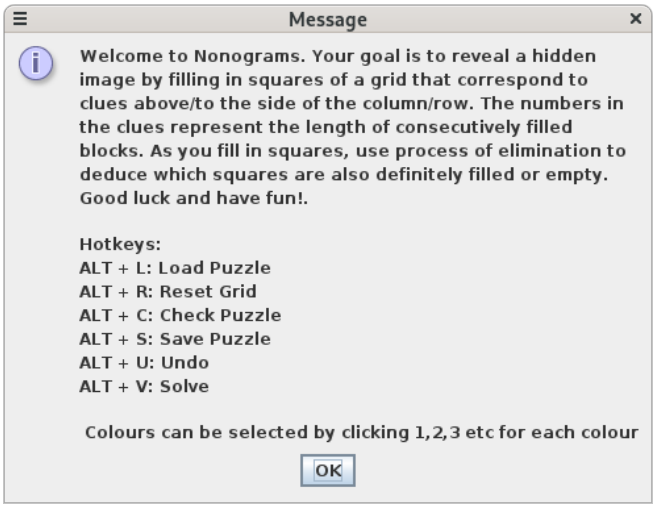


Figure 3.3: Puzzle Selection Popup

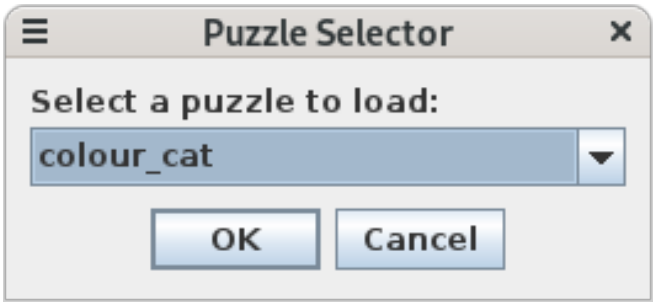


Figure 3.4: Grid Selection Popup

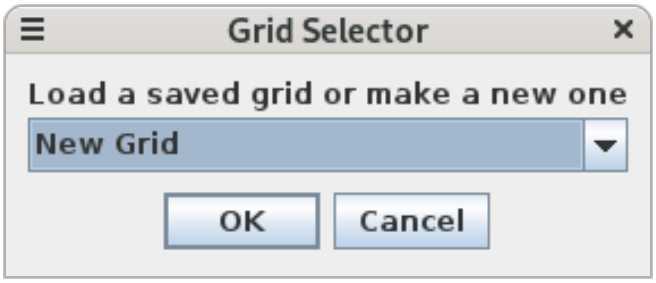


Figure 3.5: New Grid Name Popup

Figure 3.6: Main Game Screen

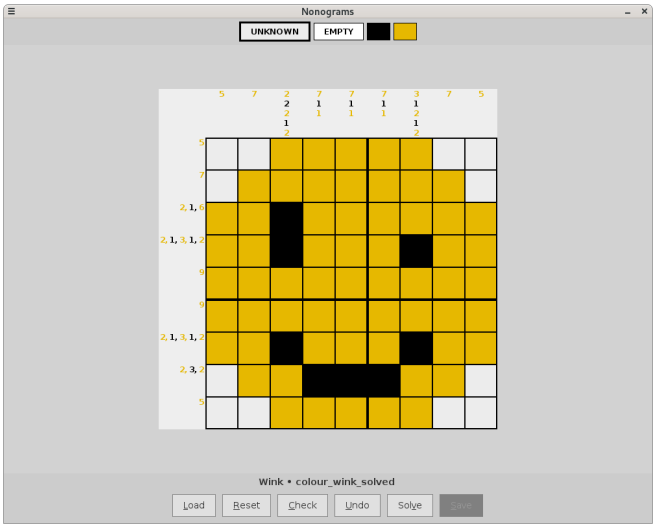


Figure 3.7: Checker Popup: Complete Puzzle

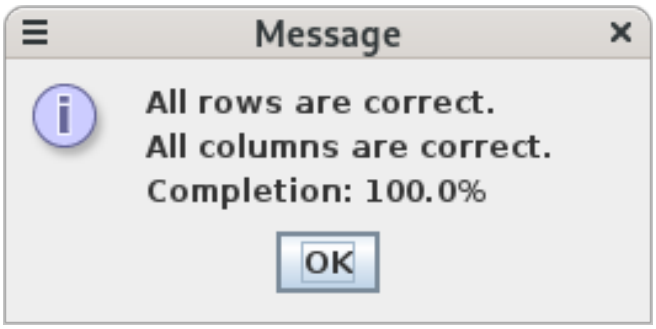
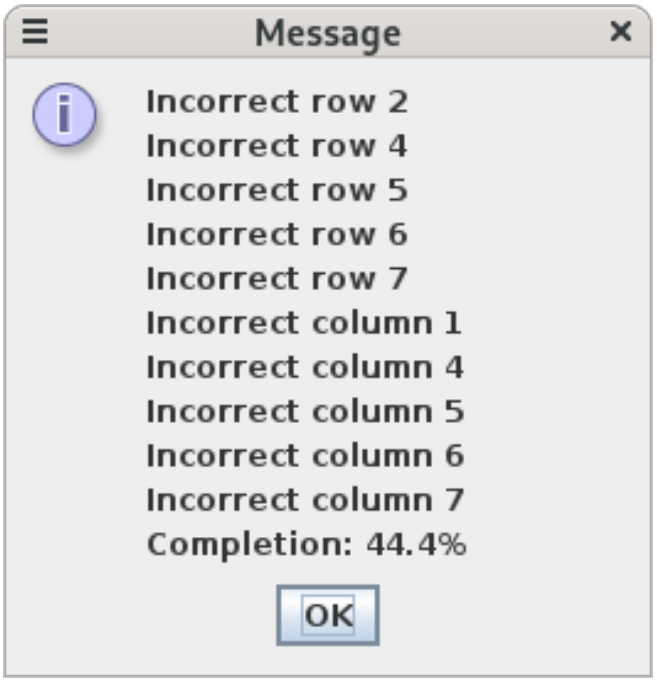


Figure 3.8: Checker Popup: Incomplete Puzzle


Message
Incorrect row 2
Incorrect row 4
Incorrect row 5
Incorrect row 6
Incorrect row 7
Incorrect column 1
Incorrect column 4
Incorrect column 5
Incorrect column 6
Incorrect column 7
Completion: 44.4%
OK